

CLASH OF THE **CLAWS**

Team: MSRIT_Hallucigen
AI Hallucination
detection platform

Abhay G. Warriar, Anirudh
Ramachandran, Dipanshu Roy,
Amogh Marthanda



Problem Statement

“AI hallucinations generates redundant/wrong answers”

As GenAI adoption explodes, enterprises blindly trust LLM outputs - but LLMs hallucinate confidently and frequently. No existing tool systematically catches these errors at scale.

LLMs hallucinate facts - fabricating citations, statistics, and events - with no built-in self-correction mechanism. Enterprise teams using AI copilots for legal drafts, medical summaries, financial reports, and research - any domain where a single wrong fact has severe consequences.

One hallucinated drug dosage in a medical AI, one false legal precedent in a contract - the damage is irreversible. Current LLMs have no factual guardrails.

Current Solution & Gaps

Existing solutions:

1. Manual fact-checking by domain experts
2. Prompt engineering ("don't make things up")
3. Basic RAG (Retrieval-Augmented Generation)

What's missing:

1. Manual checking doesn't scale with AI generation speed
 2. Prompt guardrails are unreliable - models hallucinate anyway
 3. Basic RAG retrieves context but cannot reason across multi-hop evidence or score claim-by-claim confidence
- Opportunity

A dedicated autonomous verification layer - sitting between the LLM and the user - that can decompose, search, and reason about every factual claim independently.

Your Solutions

AHDA -- Autonomous Hallucination Detection Agent

A multi-agent AI system that autonomously extracts, retrieves evidence for, and verifies every factual claim in an LLM response.

Core Ideas:

Instead of checking a paragraph as a whole, AHDA breaks it into atomic claims. Each claim is independently verified against live web sources using a 4-stage agent pipeline: Extract -> Retrieve -> Verify -> Critique.

Key Features

Multi-Agent Orchestration: Planner, Extractor, Retriever, Verifier, Critic agents working in sequence

Per-claim Confidence Scores: Every claim gets a 0-100% truthfulness score

Evidence-Backed Verdicts: Each result links to the exact source that confirms or contradicts the claim

Explainable AI: Full chain-of-thought reasoning shown to the user

Telegram Bot Integration: Send any text to the bot and receive a full Intelligence Report

Low-Latency Design: Fast Groq pipeline runs on main thread; background tasks run asynchronously

Demo/Product Walkthrough

Screen / Flow

Step 1: User opens the AHDA Dashboard (Next.js web app)

Step 2: User pastes an AI-generated response into the centered text box

Step 3: User clicks "Verify Facts" - the Agent Workflow panel shows live pipeline stages (Planner -> Extractor -> Retriever -> Verifier)

Step 4: The Intelligence Report appears below, one card per claim

Step 5: Each card shows: Verdict (TRUE/FALSE/UNVERIFIABLE), Confidence %, Reasoning, and Source Evidence

How User Interacts

The interface auto-scrolls to results when verification begins

Users can expand any claim card to see the full chain-of-thought and source links

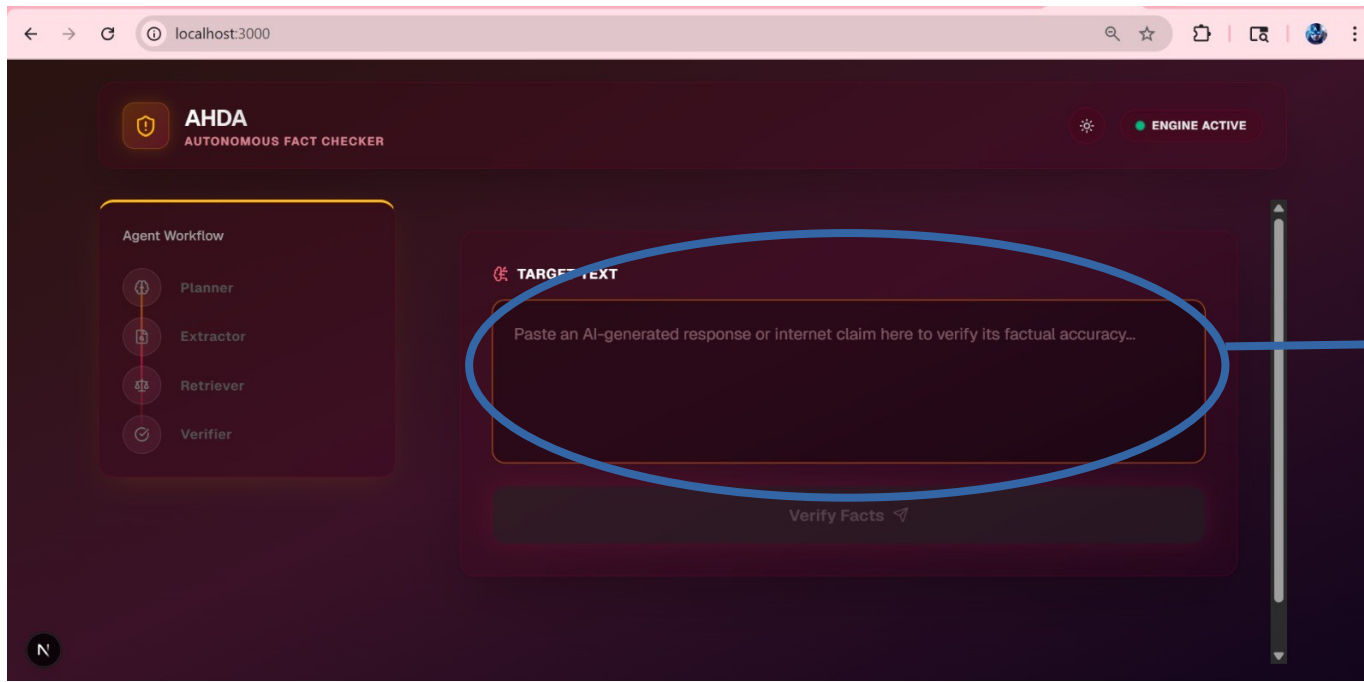
Telegram users can message the bot directly and receive a formatted report in-chat

Key Moments

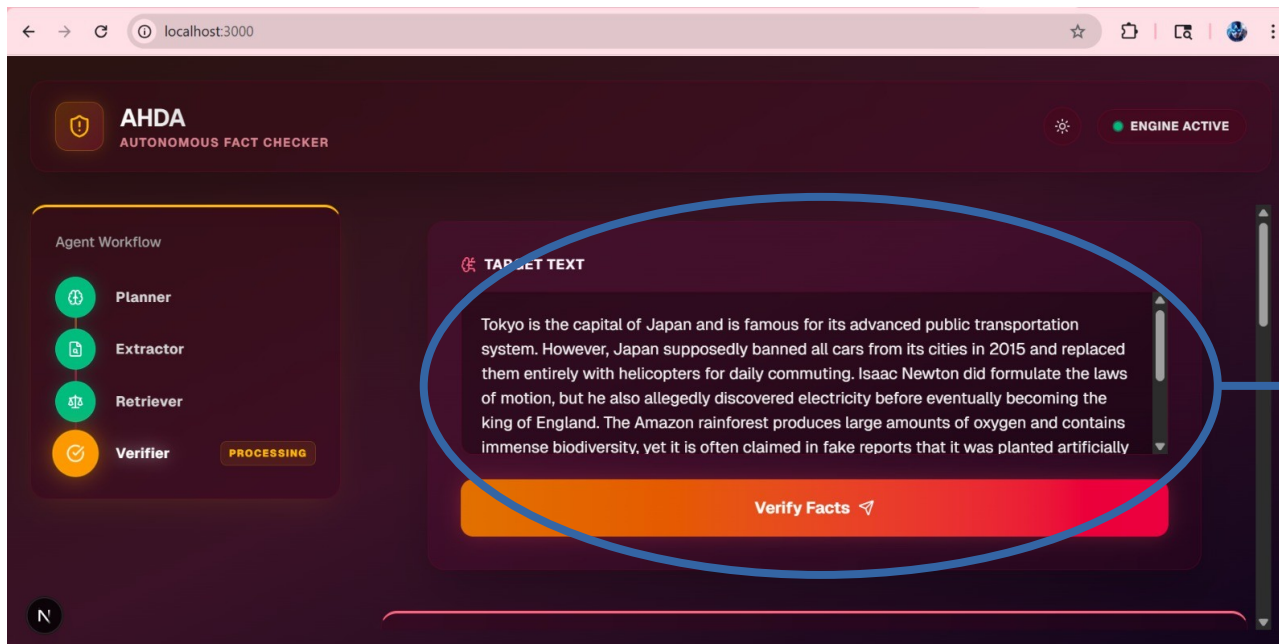
Watching the Workflow Visualizer pulse through each agent stage in real-time

Seeing a FALSE verdict fire with a confidence score and the exact contradicting source

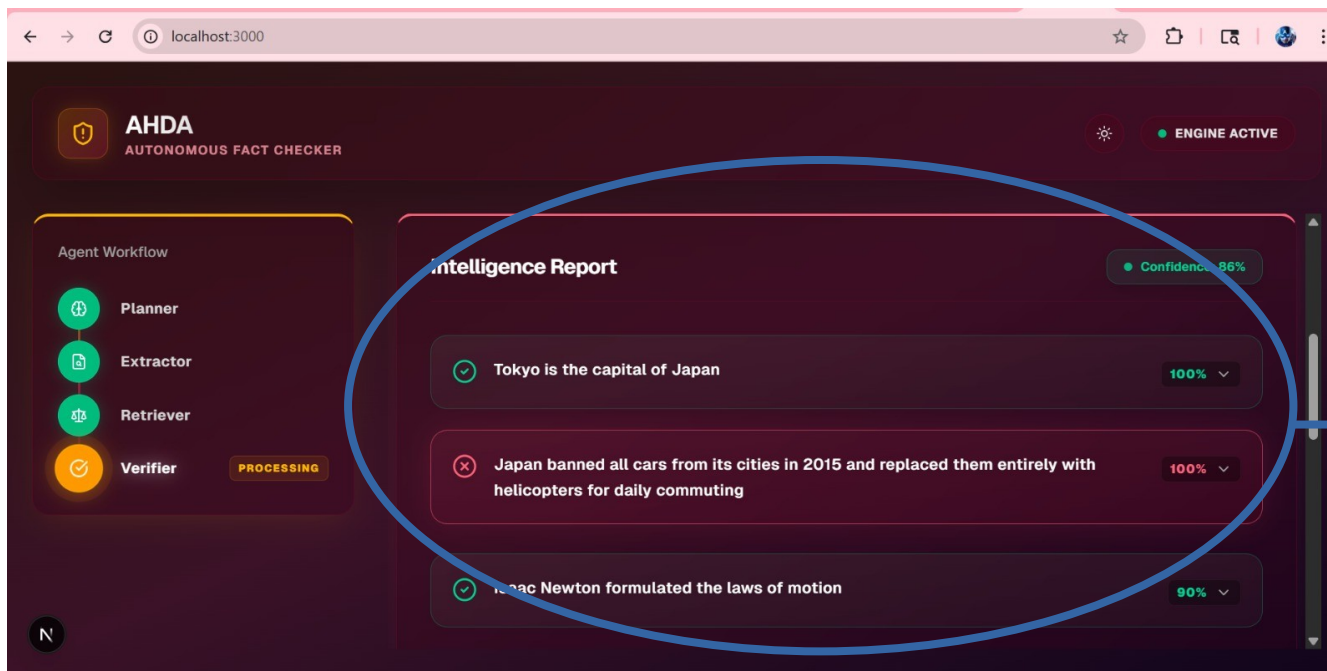
The Critic agent triggering a second search loop when initial evidence is insufficient



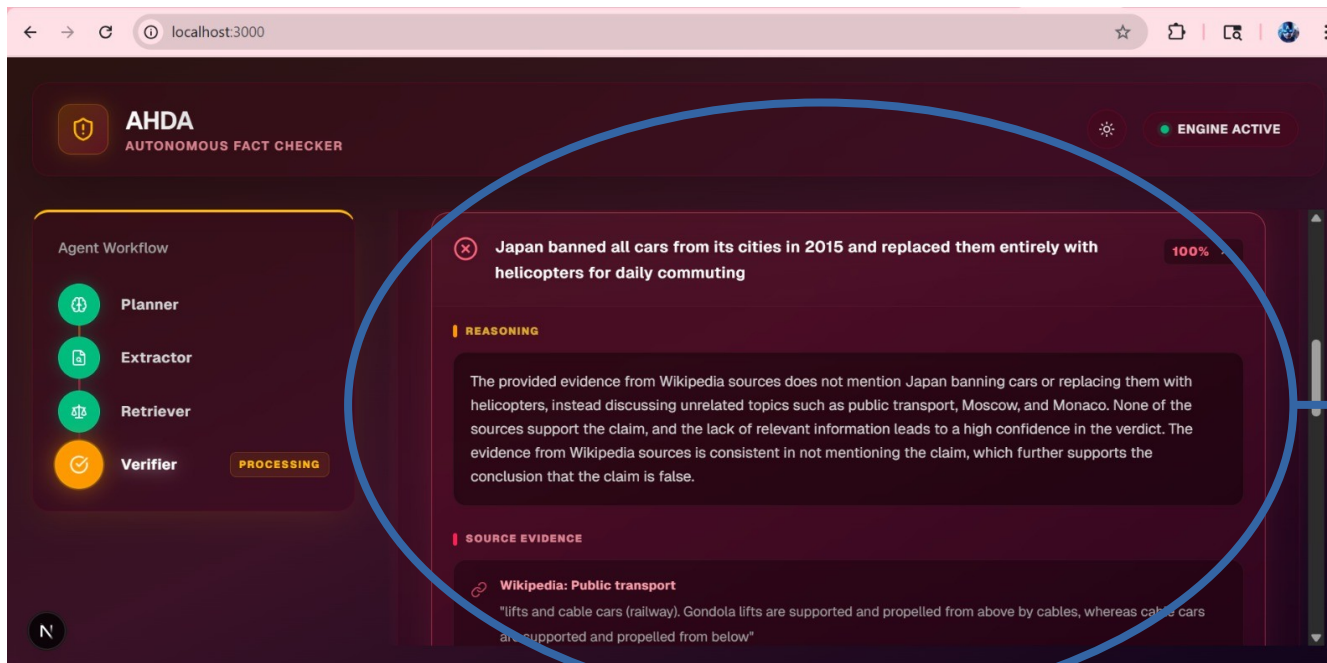
Text that needs to be verified is
put in here



Example of text that needs to
be verified



Statements after being analysed, stating if true or false with a confidence rate from 0-100%



Reasoning for statements and their citations

Tech & Architecture

Tech Stack

Frontend: Next.js 16, React, Tailwind CSS, Framer Motion

Backend: Node.js + Express (API Gateway)

AI Engine: Python skills layer (extractor, retriever, verifier)

Memory: SQLite (conversation + hallucination logs)

Messaging: Telegram Bot API (node-telegram-bot-api)

AI Usage (model, APIs, Framework)

LLM: LLaMA 3.3 via Groq API (fast inference for orchestration)

Retrieval: DuckDuckGo Search API (real-time web evidence)

Local Agent: OpenClaw (runs in background via WSL for logging/memory)

Verification Logic: Custom Chain-of-Thought prompt pipeline

System Flow (simple)

User Input -> Node.js API (/api/verify)

[Fire-and-forget] OpenClaw background log

[Main thread] Groq Pipeline:

Extractor -> Retriever (DuckDuckGo) -> Verifier

JSON response -> Frontend Intelligence Report

Impact & Use cases

Real-life Scenario 1

Legal: An AI generates a contract clause citing a court precedent that doesn't exist. AHDA flags it as FALSE with 94% confidence and links to the correct ruling.

Real-life Scenario 2

Healthcare: A clinical AI summarizes a patient's medication history with a wrong dosage. AHDA intercepts, retrieves drug reference data, and flags the discrepancy before it reaches the doctor.

Real-life Scenario 3

Finance: An AI-generated earnings report summary cites incorrect quarterly figures. AHDA cross-checks against live financial data and marks the claim UNVERIFIABLE, triggering a manual review flag.

Real-life Scenario 4

Education/Research: A student uses an AI to generate a research bibliography. AHDA verifies each citation, flagging 3 out of 10 as hallucinated references that don't exist in any database.

Scale Potential

AHDA can be deployed as a middleware API firewall in front of any enterprise chatbot - requiring zero changes to the underlying LLM. At scale, it becomes the factual immune system for all AI outputs organization-wide.

Differentiation

AHDA doesn't just retrieve context like RAG - it reasons. A dedicated Critic agent reflects on the verifier's output, identifies weak evidence, and autonomously triggers refined secondary searches until a high-confidence verdict is reached.

Multi-Agent Reasoning Loop: The Critic -> Retriever feedback cycle means AHDA self-improves its own evidence within a single request. No other open tool does this autonomously per-claim.

Hybrid Latency Architecture: A fire-and-forget design separates the user-facing fast Groq pipeline from the deep OpenClaw background execution - delivering sub-2-second responses while comprehensive logging runs silently in parallel. Users never wait.

Demo Link

GitHub Repo: <https://github.com/abhay-g-w-hub/AHDA-by-Hallucigen>

Demo Link: <https://www.youtube.com/watch?v=AhTp5TsGe9o>

Document submitted:

https://github.com/abhay-g-w-hub/AHDA-by-Hallucigen/blob/master/OpenClaw_AI_Disclosure.docx